

在接下来的几章中，我们将讨论一组重要的并行计算模式。这些模式是许多并行应用中出现的并行算法的基础。我们将从卷积开始，它是一种流行的数组操作，在信号处理、数字录音、图像处理、视频处理和计算机视觉中以各种形式使用。在这些应用领域中，卷积通常作为一种滤波器进行，将信号和像素转换为更理想的值。我们的图像模糊核就是这样一种滤波器，它可以平滑信号值，使人们能够看到大局趋势。另一个例子是高斯滤波器，它是一种卷积滤波器，可以用来增强图像中物体的边界和边缘。

卷积通常需要执行大量的算术运算来生成每个输出元素。对于大型数据集，比如高清图像和视频，其中有许多输出元素（像素），计算量可能会非常庞大。一方面，卷积的每个输出数据元素可以独立计算，这是并行计算中的一个优点。另一方面，在处理不同输出数据元素时存在大量的输入数据共享，且边界条件有些复杂。这使得卷积成为复杂平铺方法和输入数据分阶段方法的重要应用案例，这也是本章的重点讨论内容。

7.1 背景

卷积是一种数组操作，其中每个输出数据元素是对应输入元素和以其为中心的一组输入元素的加权和。用于加权和计算的权重由一个滤波器数组定义，通常称为卷积核。由于CUDA核函数和卷积核之间存在不幸的名称冲突，我们将这些滤波器数组称为卷积滤波器，以避免混淆。

卷积可以在不同维度的输入数据上执行：一维（1D）（例如音频）、二维（2D）（例如照片）、三维（3D）（例如视频）等等。在音频数字信号处理中，输入的一维数组元素是随时间采样的信号音量。也就是说，输入数据元素 x_i 是音频信号音量的第 i 个采样。对于一维数据的卷积，称为1D卷积，数学上定义为一个接受 n 个元素 $[x_0, x_1, \dots, x_{n-1}]$ 的输入数据数组和一个 $2r + 1$ 个元素 $[f_0, f_1, \dots, f_{2r}]$ 的滤波器数组，并返回一个输出数据数组 y ：

$$y_i = \sum_{j=-r}^r f_{j+r} \times x_{i+j}$$

由于滤波器的大小是奇数（ $2r + 1$ ），加权和计算在正在计算的元素周围是对称的。也就是说，加权和涉及到正在计算的位置周围 r 个输入元素，这就是为什么 r 被称为滤波器的半径的原因。

图7.1展示了一个一维卷积的例子，其中应用了一个五个元素（ $r = 2$ ）的卷积滤波器 f 到一个七个元素的输入数组 x 。我们遵循C语言的惯例，其中 x 和 y 元素从0到6索引， f 元素从0到4索引。由于滤波器半径为2，每个输出元素被计算为对应输入元素、左侧两个元素和右侧两个元素的加权和。

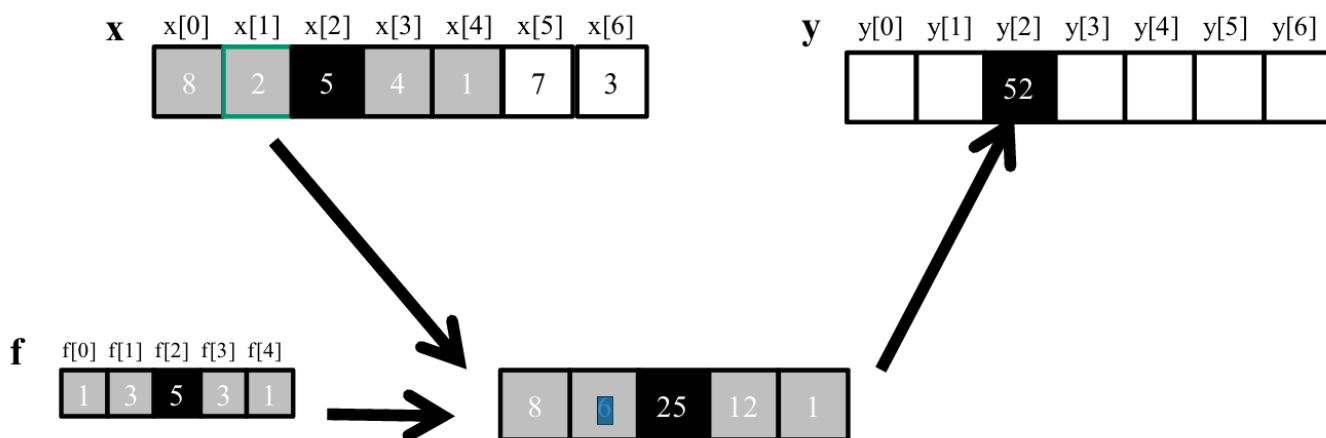


图7.1 一维卷积示例，内部元素

例如， $y[2]$ 的值是由 $x[0]$ （ $x[2 - 2]$ ）到 $x[4]$ （ $x[2 + 2]$ ）的加权和生成的。在这个例子中，我们任意假设 x 元素的值为 $[8, 2, 5, 4, 1, 7, 3]$ 。 f 元素定义了权重，在这个例子中的值分别为1、3、5、3、1。每个 f 元素在将产品求和之前都与对应的 x 元素值相乘。如图7.1所示， $y[2]$ 的计算如下：

$$\begin{aligned}
 y[2] &= f[0]*x[0] + f[1]*x[1] + f[2]*x[2] + f[3]*x[3] + f[4]*x[4] \\
 &= 1*8 + 3*2 + 5*5 + 3*4 + 1*1 \\
 &= 52
 \end{aligned}$$

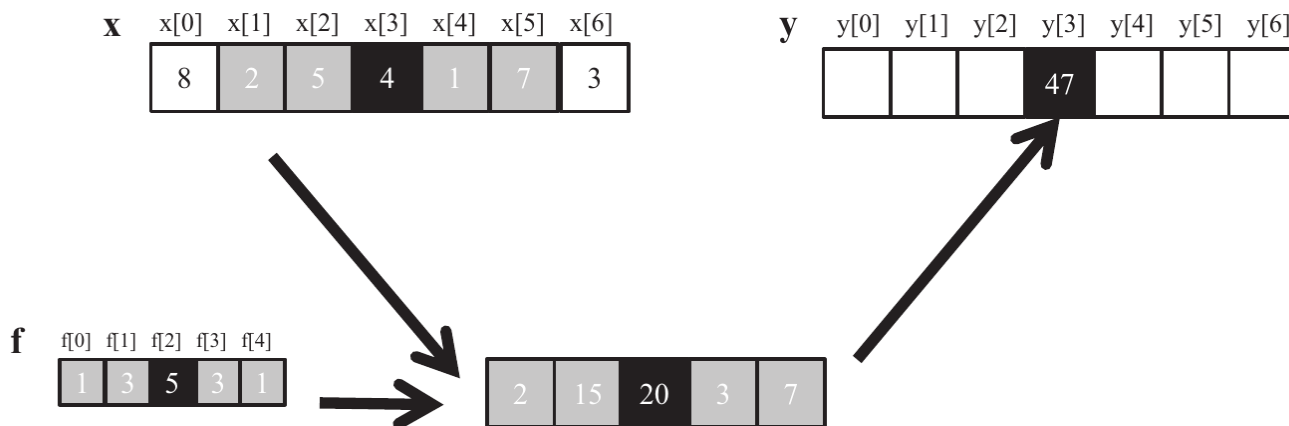


图7.2 一维卷积示例，计算 $y[3]$

在图7.1中， $y[i]$ 的计算可以看作是 x 从 $x[i-2]$ 开始的子数组与 f 数组之间的内积。图7.2展示了 $y[3]$ 的计算过程。该计算相对于图7.1的计算向右移动了一个 x 元素。也就是说， $y[3]$ 的值是由 $x[1]$ ($x[3-2]$) 到 $x[5]$ ($x[3+2]$) 的加权和生成的。我们可以将 $x[3]$ 的计算看作如下内积：

$$\begin{aligned}
 y[3] &= f[0]*x[1] + f[1]*x[2] + f[2]*x[3] + f[3]*x[4] + f[4]*x[5] \\
 &= f[0]*x[1] + f[1]*x[2] + f[2]*x[3] + f[3]*x[4] + f[4]*x[5] \\
 &= 1*2 + 3*5 + 5*4 + 3*1 + 1*7 \\
 &= 47
 \end{aligned}$$

由于卷积是根据相邻元素定义的，因此在计算接近数组末端的输出元素时会自然产生边界条件。如图7.3所示，当我们计算 $y[1]$ 时，在 $x[1]$ 的左侧只有一个 x 元素。也就是说，根据我们对卷积的定义，没有足够的 x 元素来计算 $y[1]$ 。处理这种边界条件的一种典型方法是为这些缺失的 x 元素赋一个默认值。对于大多数应用程序，该默认值是0，在图7.3中我们使用了这个默认值。例如，在音频信号处理中，我们可以假设在录音开始之前和结束之后，信号音量为0。在这种情况下， $y[1]$ 的计算如下：

$$\begin{aligned}
 y[1] &= f[0]*0 + f[1]*x[0] + f[2]*x[1] + f[3]*x[2] + f[4]*x[3] \\
 &= 1*0 + 3*8 + 5*2 + 3*5 + 1*4 \\
 &= 53
 \end{aligned}$$

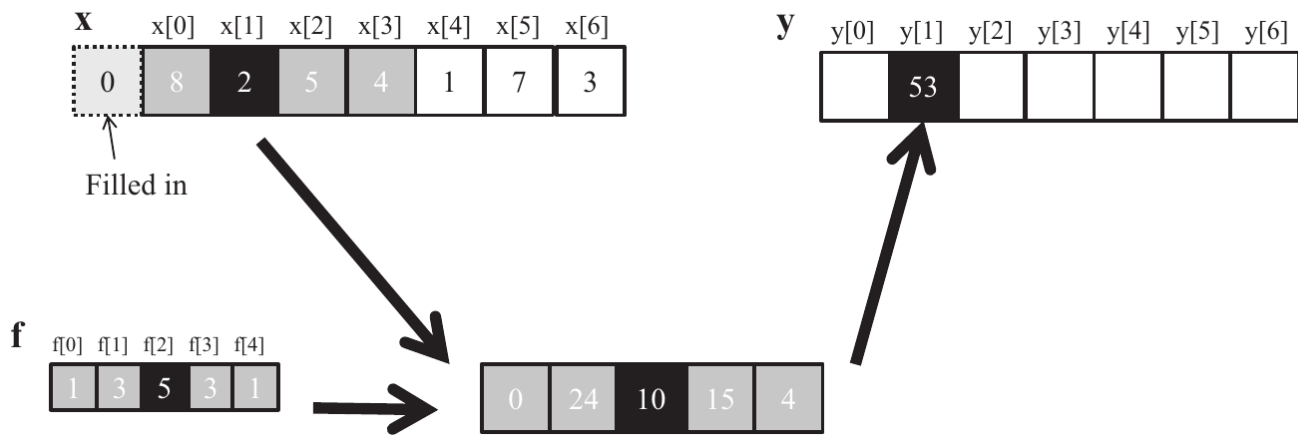


图7.3 一维卷积边界条件

在图7.3中，不存在的x元素用虚线框表示。很明显， $y[0]$ 的计算将涉及两个缺失的x元素，这两个元素在本例中都被假定为0。我们将 $y[0]$ 的计算留作练习。这些缺失的元素通常在文献中被称为“幽灵单元”(ghost cell)。由于在并行计算中使用平铺，还会出现其他类型的幽灵单元。这些幽灵单元可能对平铺的有效性和/或效率产生重大影响。我们将很快回到这一点。

此外，并非所有应用程序都假设幽灵单元包含0。例如，某些应用程序可能假设幽灵单元包含与边缘上最接近的有效数据元素相同的值。

对于图像处理和计算机视觉，输入数据通常表示为二维数组，在x-y空间中表示像素。因此，图像卷积也是二维卷积，如图7.4所示。在二维卷积中，滤波器f也是一个二维数组。它的x和y维度确定了在加权和计算中应包括的邻居范围。如果我们假设滤波器的尺寸在x维度为 $(2r_x + 1)$ ，在y维度为 $(2r_y + 1)$ ，那么每个P元素的计算可以表示如下：

$$P_{y,x} = \sum_{j=-r_y}^{r_y} \sum_{k=-r_x}^{r_x} f_{j+r_y, k+r_x} N_{y+k, x+j}$$

在图7.4中，为简单起见，我们使用了一个 5×5 的滤波器；即 $r_y = 2, r_x = 2$ 。一般来说，滤波器不必是正方形数组，但通常是这样的。为了生成一个输出元素，我们取以输入数组N中对应位置为中心的子数组。然后，我们在滤波器数组和图像数组之间进行逐对乘法。对于我们的例子，结果显示为图7.4中N和P下方的 5×5 乘积数组。输出元素的值是乘积数组的所有元素的和。

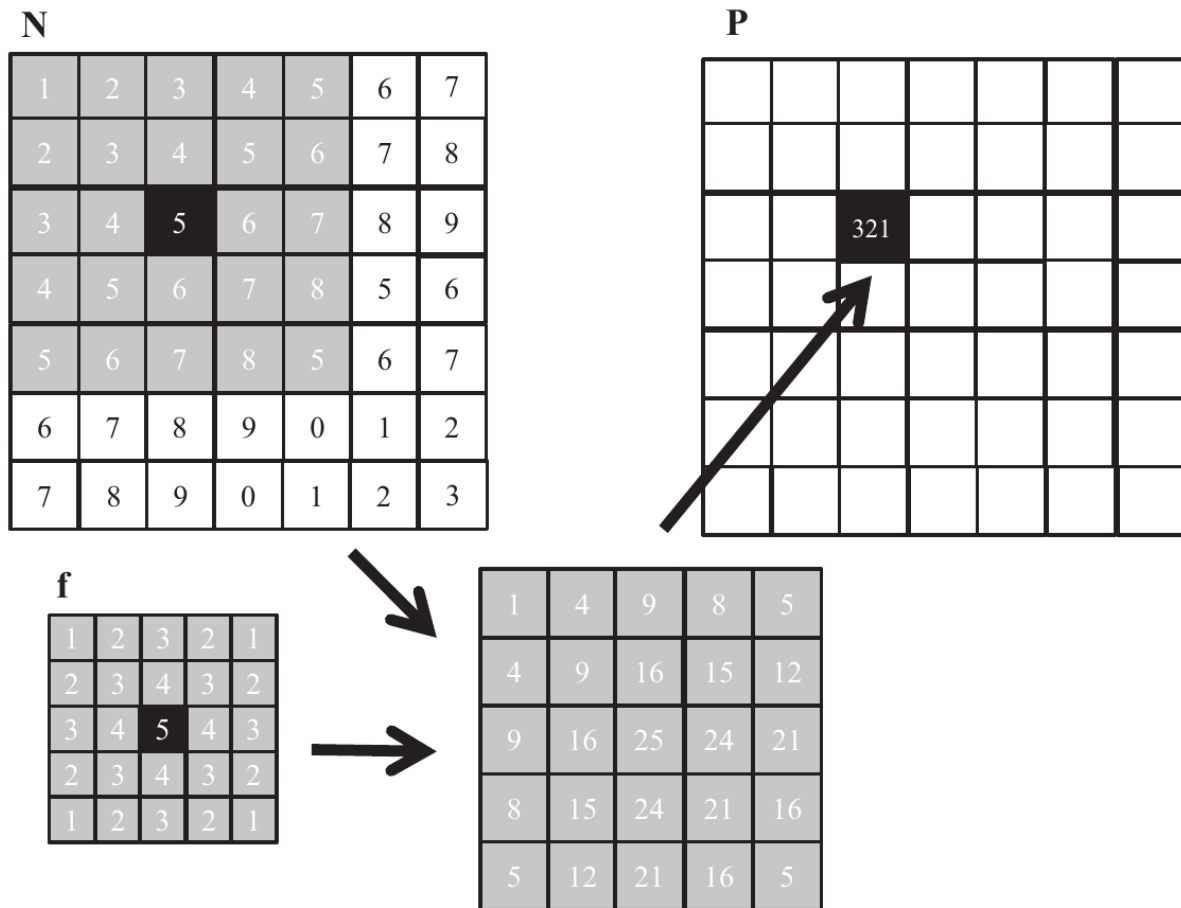


图7.4 二维卷积的例子

图7.4中的示例展示了 $P_{2,2}$ 的计算过程。为了简洁起见，在我们实际的代码示例中，我们将使用 $N_{\{y,x\}}$ 来表示在C数组中寻址时的 $N[y][x]$ 。由于N和P很可能是动态分配的数组，我们将在实际代码中使用线性化索引。计算如下：

$$\begin{aligned}
 P_{2,2} &= N_{0,0} * M_{0,0} + N_{0,1} * M_{0,1} + N_{0,2} * M_{0,2} + N_{0,3} * M_{0,3} + N_{0,4} * M_{0,4} \\
 &\quad + N_{1,0} * M_{1,0} + N_{1,1} * M_{1,1} + N_{1,2} * M_{1,2} + N_{1,3} * M_{1,3} + N_{1,4} * M_{1,4} \\
 &\quad + N_{2,0} * M_{2,0} + N_{2,1} * M_{2,1} + N_{2,2} * M_{2,2} + N_{2,3} * M_{2,3} + N_{2,4} * M_{2,4} \\
 &\quad + N_{3,0} * M_{3,0} + N_{3,1} * M_{3,1} + N_{3,2} * M_{3,2} + N_{3,3} * M_{3,3} + N_{3,4} * M_{3,4} \\
 &\quad + N_{4,0} * M_{4,0} + N_{4,1} * M_{4,1} + N_{4,2} * M_{4,2} + N_{4,3} * M_{4,3} + N_{4,4} * M_{4,4} \\
 &= 1 * 1 + 2 * 2 + 3 * 3 + 4 * 2 + 5 * 1 \\
 &\quad + 2 * 2 + 3 * 3 + 4 * 4 + 5 * 3 + 6 * 2 \\
 &\quad + 3 * 3 + 4 * 4 + 5 * 5 + 6 * 4 + 7 * 3 \\
 &\quad + 4 * 2 + 5 * 3 + 6 * 4 + 7 * 3 + 8 * 2 \\
 &\quad + 5 * 1 + 6 * 2 + 7 * 3 + 8 * 2 + 5 * 1 \\
 &= 1 + 4 + 9 + 8 + 5 \\
 &\quad + 4 + 9 + 16 + 15 + 12 \\
 &\quad + 9 + 16 + 25 + 24 + 21 \\
 &\quad + 8 + 15 + 24 + 21 + 16 \\
 &\quad + 5 + 12 + 21 + 16 + 5 \\
 &= 321
 \end{aligned}$$

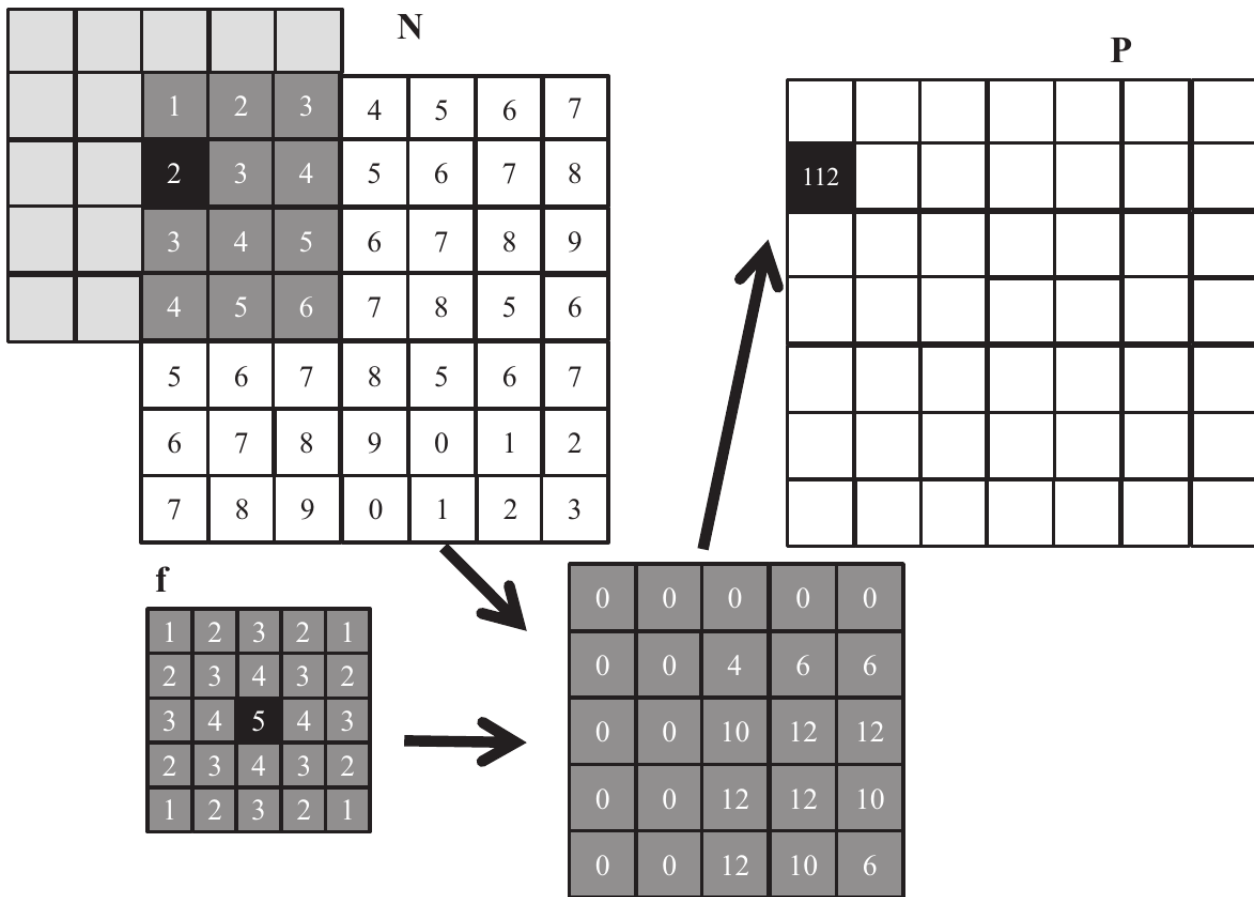


图7.5 二维卷积的边界条件

与一维卷积一样，二维卷积也必须处理边界条件。在x和y维度上都有边界时，边界条件更加复杂：计算输出元素可能涉及水平边界、垂直边界或两者的边界条件。图7.5展示了涉及两个边界的P元素计算过程。从图7.5可以看出， $P_{1,0}$ 的计算涉及到N子数组中两个缺失的列和一个缺失的行。与一维卷积一样，不同的应用程序对于这些缺失的N元素假设不同的默认值。在我们的示例中，我们假设默认值为0。这些边界条件也会影响平铺的效率。我们将很快回到这一点。

7.2 并行卷积：基本算法

卷积中所有输出元素的计算都可以并行进行，这使得卷积成为并行计算的理想应用案例。基于我们在矩阵乘法中的经验，我们可以快速编写一个简单的并行卷积核。我们将展示2D卷积的代码示例，并鼓励读者将这些代码示例调整为1D和3D的练习。此外，为简单起见，我们假设滤波器是正方形的。

第一步是为卷积核定义主要的输入参数。我们假设2D卷积核接收五个参数：指向输入数组N的指针；指向滤波器F的指针；指向输出数组P的指针；正方形滤波器的半径r；输入和输出数组的宽度width；以及输入和输出数组的高度height。因此我们有以下设置：

```
__global__ void
convolution_2D_basic_kernel(float *N, float *F, float *P, int r,
                           int width, int height) {
    // kernel body
}
```

第二步是确定并实现线程与输出元素的映射关系。由于输出数组是2D的，一个简单而好的方法是将线程组织成一个2D网格，每个网格中的线程计算一个输出元素。每个块可以包含最多1024个线程，并且每个块可以计算最多1024个输出元素。图7.6展示了一个玩具示例，其中输入和输出是 16×16 的图像。在这个玩具示例中，我们

假设每个线程块组织成一个 4×4 的线程数组：x维度有四个线程，y维度也有四个线程。在这个例子中，网格被组织成一个 4×4 的块数组。线程与输出元素（本例中是输出像素）的分配很简单：每个线程被分配到计算一个与其x和y索引相同的输出像素。

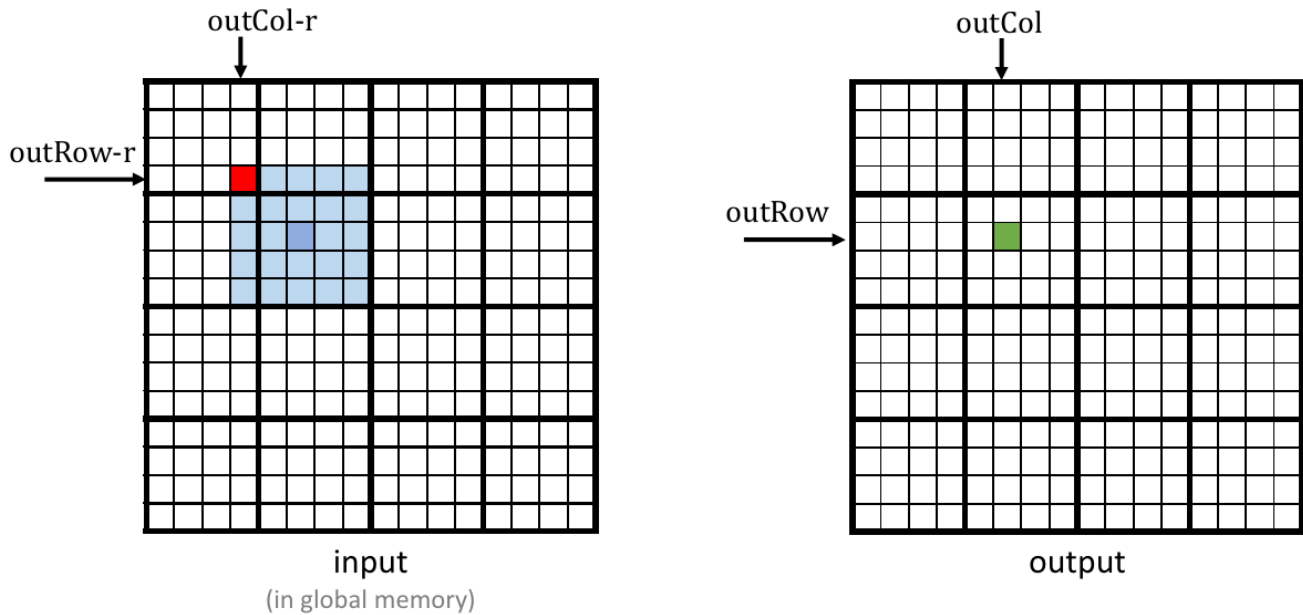


图7.6 二维卷积的并行和线程组织

读者应该注意，图7.6中的并行化安排与第3章“多维网格和数据”中的ColorToGrayScaleConversion示例相同。因此，我们可以使用图7.7中卷积核的02和03行的语句来计算每个线程的输出元素索引，这些语句是从块索引、块维度和线程索引计算出来的。例如， $\text{block}\{1,1\}$ 的 $\text{thread}\{1,1\}$ 被映射到输出元素 $P[1 * 4 + 1][1 * 4 + 1] = P[5][5]$ ，在图7.6中标记为绿色方块。

确定了每个线程的输出元素索引之后，我们可以确定计算输出元素所需的输入N元素。如图7.6所示，通过 $\text{block}\{1,1\}$ 的 $\text{thread}\{1,1\}$ 计算 $P[5][5]$ （绿色方块）将使用x索引范围从 $\text{outCol} - r = 3$ 到 $\text{outCol} + r = 7$ ，以及y索引范围从 $\text{outRow} - r = 3$ 到 $\text{outRow} + r = 7$ 的输入元素。对于所有线程， $\text{outCol} - r$ 和 $\text{outRow} - r$ 定义了用于 $P[\text{outRow}][\text{outCol}]$ 的输入元素区域的左上角（浓重阴影方块）和轻微阴影区域。因此，我们可以使用双重嵌套循环来遍历所有这些索引值并执行此计算（图7.7的05-13行）。

寄存器变量Pvalue将累积所有中间结果以节省DRAM带宽。内部for循环中的if语句测试用于计算输出元素的输入N元素是否位于N数组的左侧、右侧、顶部或底部的幽灵单元。由于我们假设幽灵单元的值为0，因此我们可以简单地跳过幽灵单元元素及其对应的滤波器元素的乘法和累加。循环结束后，我们将Pvalue释放到输出P元素中（图7.7的第14行）。

```

01 __global__ void convolution_2D_basic_kernel(float *N, float *F, float *P,
    int r, int width, int height) {
02     int outCol = blockIdx.x*blockDim.x + threadIdx.x;
03     int outRow = blockIdx.y*blockDim.y + threadIdx.y;
04     float Pvalue = 0.0f;
05     for (int fRow = 0; fRow < 2*r+1; fRow++) {
06         for (int fCol = 0; fCol < 2*r+1; fCol++) {
07             inRow = outRow - r + fRow;
08             inCol = outCol - r + fCol;
09             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
10                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
11             }
12         }
13     }
14     P[outRow][outCol] = Pvalue;
15 }

```

图7.7 考虑了边界条件的二维卷积kernel

我们对图7.7中的卷积核进行两点观察。首先，存在控制流分歧。计算P数组四个边缘附近输出元素的线程需要处理幽灵单元。正如我们在第7.1节中所示，这些线程中的每一个将遇到不同数量的幽灵单元。因此，它们在if语句（第9行）中都会有不同的决策。计算P[0][0]的线程大部分时间将跳过乘积累加语句，而计算P[0][1]的线程将跳过的次数较少，依此类推。控制流分歧的成本取决于输入数组的宽度和高度以及滤波器的半径。对于大型输入数组和小型滤波器，控制流分歧仅在计算输出元素的一小部分时发生，这将使控制流分歧的影响保持较小。由于卷积通常应用于大型图像，我们期望控制流分歧的影响从适度到微不足道。

一个更严重的问题是内存带宽。浮点运算计算与全局内存访问的比率仅约为0.25 OP/B（第10行加载的每8字节进行2次操作）。就像我们在矩阵乘法示例中看到的那样，这个简单的核心只能期望以极小的一部分峰值性能运行。接下来的两节中，我们将讨论两种减少全局内存访问的技术。

7.3 常量内存和缓存

在卷积中，过滤器数组F的使用方式有三个有趣的特性。首先，F的大小通常很小；大多数卷积滤波器的半径为7或更小。即使在三维卷积中，滤波器通常也只包含小于或等于 $7^3 = 343$ 个元素。其次，F的内容在卷积核的执行过程中不会改变。第三，所有线程都会访问滤波器元素。更好的是，所有线程以相同的顺序访问F元素，从F[0][0]开始，并且通过图7.7中双重嵌套的for循环迭代逐个元素移动。这三个特性使得该滤波器成为常量内存和缓存的优秀候选者（图7.8）。

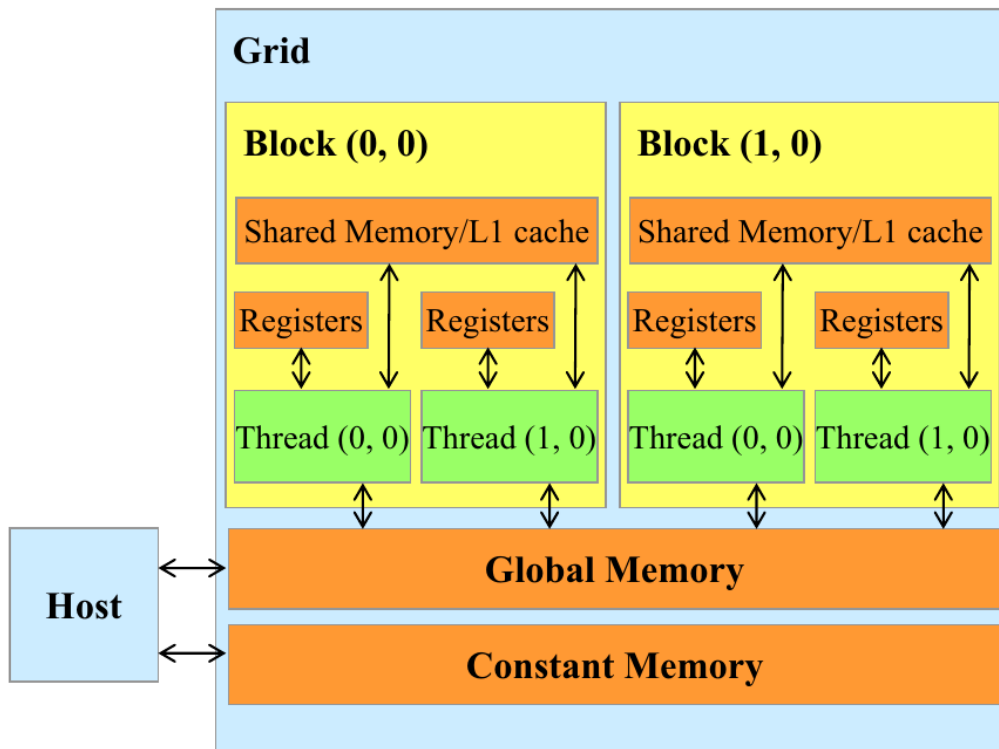


图7.8 CUDA内存模型

正如我们在第 5 章中讨论的《内存架构和数据局部性》(表 5.1) 中所述，CUDA C 允许程序员声明变量驻留在常量内存中。与全局内存变量类似，常量内存变量对所有线程块可见。主要区别在于，在内核执行过程中，常量内存变量的值不能被线程修改。此外，常量内存的大小相当小，目前为 64 KB。

要使用常量内存，主机代码需要以与全局内存变量不同的方式分配和复制常量内存变量。我们假设滤波器的半径在编译时常量 `FILTER_RADIUS` 中指定。要在常量内存中声明一个 `F` 数组，主机代码应该将其声明为全局变量，如下所示：

```
__constant__ float F[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
```

请注意，这是一个全局变量声明，应该在源文件中的任何函数之外。关键字 `__constant__` (两侧各两个下划线) 告诉编译器数组 `F` 应该放置在设备常量内存中。假设主机代码已经在主机内存中的一个大小为 $(2 \times \text{FILTER_RADIUS} + 1)^2$ 元素的滤波器 `F_h` 数组中分配并初始化了蒙版的内容。`F_h` 的内容可以从主机内存传输到设备常量内存中的 `F` 如下所示：

```
cudaMemcpyToSymbol(F, F_h, (2 * FILTER_RADIUS + 1) * (2 * FILTER_RADIUS + 1) *
    sizeof(float));
```

请注意，这是一个特殊的内存复制函数，通知 CUDA 运行时正在复制到常量内存的数据在内核执行期间不会被更改。一般来说，使用 `cudaMemcpyToSymbol()` 函数的方式如下：

```
cudaMemcpyToSymbol(dest, src, size);
```


其中 `dest` 是指向常量内存中目标位置的指针，`src` 是指向主机内存中源数据的指针，`size` 是要复制的字节数。

内核函数访问常量内存变量就像访问全局变量一样。因此，它们的指针不需要作为参数传递给内核。我们可以修改我们的内核以使用常量内存，如图 7.9 所示。请注意，该内核几乎与图 7.7 中的内核相同。唯一的区别是 `F` 不再通过作为参数传递的指针访问。现在它作为一个全局变量访问。请记住，所有的 C 语言作用域规则都适用于全局变量。如果主机代码和内核代码在不同的文件中，内核代码文件必须包含相关的外部声明信息，以确保 `F` 的声明对内核可见。

```
01 __global__ void convolution_2D_const_mem_kernel(float *N, float *P, int r,
    int width, int height) {
02     int outCol = blockIdx.x*blockDim.x + threadIdx.x;
03     int outRow = blockIdx.y*blockDim.y + threadIdx.y;
04     float Pvalue = 0.0f;
05     for (int fRow = 0; fRow < 2*r+1; fRow++) {
06         for (int fCol = 0; fCol < 2*r+1; fCol++) {
07             inRow = outRow - r + fRow;
08             inCol = outCol - r + fCol;
09             if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
10                 Pvalue += F[fRow][fCol]*N[inRow*width + inCol];
11             }
12         }
13     }
14     P[outRow*width+outCol] = Pvalue;
15 }
```

图7.9 使用常量内存的二维卷积核对 `F` 进行操作

与全局内存变量一样，常量内存变量也位于 DRAM 中。然而，由于 CUDA 运行时知道常量内存变量在内核执行期间不会被修改，它会指示硬件在内核执行期间积极缓存常量内存变量。要理解常量内存使用的好处，我们首先需要更多地了解现代处理器的内存和缓存层次结构。

正如我们在第 6 章中讨论的《性能考虑》中所述，DRAM 的长延迟和有限带宽在几乎所有现代处理器中都构成瓶颈。为了缓解这种内存瓶颈的影响，现代处理器通常使用片上缓存内存，或缓存，来减少需要从主内存（DRAM）访问的变量数量，如图 7.10 所示。

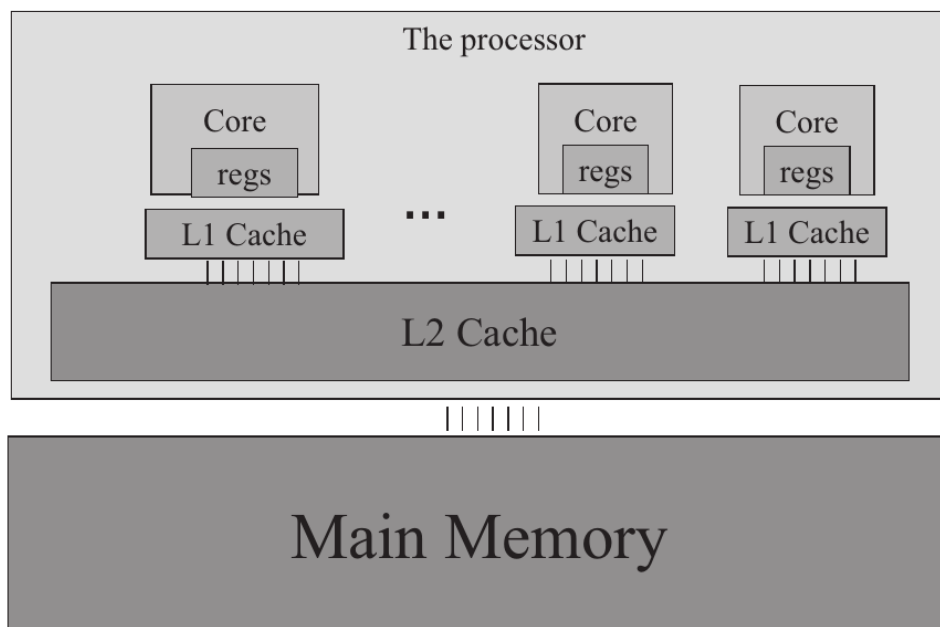


图7.10 现代处理器缓存层次结构的简化视图。

与 CUDA 共享内存或一般的内存不同，缓存对程序是“透明”的。也就是说，要使用 CUDA 共享内存来保存全局变量的值，程序需要将变量声明为 `__shared__` 并将全局内存变量的值显式复制到共享内存变量中。另一方面，

在使用缓存时，程序只需访问原始的全局内存变量。处理器硬件将自动将最近或最频繁使用的变量保留在缓存中，并记住它们的原始全局内存地址。当稍后使用保留的变量之一时，硬件将从其地址中检测到缓存中存在该变量的副本。然后，变量的值将从缓存中提供，无需访问 DRAM。

内存的大小和速度之间存在权衡。因此，现代处理器通常使用多级缓存。这些缓存级别的编号约定反映了与处理器的距离。最低级别，L1 或级别 1，是直接连接到处理器核心的缓存，如图 7.10 所示。它在延迟和带宽方面的速度接近处理器。然而，L1 缓存很小，通常容量在 16 到 64 KB 之间。L2 缓存较大，容量在几百 KB 到几 MB 之间，但访问需要十几个周期。它们通常在多个处理器核心之间共享，或在 CUDA 设备中的流式多处理器 (SMs) 之间共享，因此访问带宽在 SMs 之间共享。在今天的一些高端处理器中，甚至还有可以达到数百 MB 大小的 L3 缓存。

常量内存变量在设计和使用大规模并行处理器内存时发挥着有趣的作用。由于这些常量内存变量在内核执行期间不会被修改，因此在 SM 中缓存时不需要支持线程的写入。支持将数据写入通用缓存需要复杂的硬件逻辑，并且在芯片面积和功耗方面成本高昂。无需支持写入，可以以高效的方式设计专用于常量内存变量的缓存，以节省芯片面积和功耗。此外，由于常量内存相当小 (64 KB)，因此一个小型的专用缓存可以高效地捕获每个内核中使用频繁的常量内存变量。这种专用缓存在现代 GPU 中被称为常量缓存。因此，当一个 warp 中的所有线程访问相同的常量内存变量时，如图 7.9 中的 F，其中用于访问 F 的索引独立于线程索引，常量缓存可以提供巨大的带宽，以满足这些线程的数据需求。此外，由于常量内存相当小 (64 KB)，因此针对每个内核的高频使用的常量内存变量，一个小型的专用缓存可以非常有效地捕获这些变量。这种专用缓存在现代 GPU 中称为常量缓存。因此，当一个 warp 中的所有线程访问相同的常量内存变量时，就像图 7.9 中的 F 一样，其中访问 F 的索引与线程索引无关，常量缓存可以提供大量带宽以满足这些线程的数据需求。另外，由于 F 的大小通常很小，我们可以假设所有 F 元素始终有效地从常量缓存中访问。因此，我们可以简单地假设对 F 元素的访问不会花费任何 DRAM 带宽。通过使用常量内存和缓存，我们有效地将浮点运算与内存访问的比率增加了一倍，达到了约 0.5 OP/B (每加载 4 个字节的第 10 行进行 2 次操作)。

事实证明，对输入 N 数组元素的访问也可以受益于缓存。我们将在第 7.5 节回到这一点。

7.4 具有halo单元的瓦片卷积

我们可以通过使用瓦片卷积算法来解决卷积的内存带宽瓶颈。回想一下，在瓦片算法中，线程合作将输入元素加载到片上内存中，以供后续使用这些元素。我们将首先确定输入和输出瓦片的定义，因为这些定义对于理解算法设计非常重要。我们将把每个块处理的输出元素集合称为输出瓦片。回想一下，图 7.6 展示了使用 16 个块，每个块有 16 个线程的玩具示例的 16×16 二维卷积。在该示例中，有 16 个输出瓦片。请注意，为了保持示例的简洁性，我们每个块使用 16 个线程。在实践中，每个块应该至少有 32 个线程，或者一个 warp，并且通常有更多线程以实现良好的占用率和数据重用。从此处开始，我们假设 F 元素位于常量内存中。

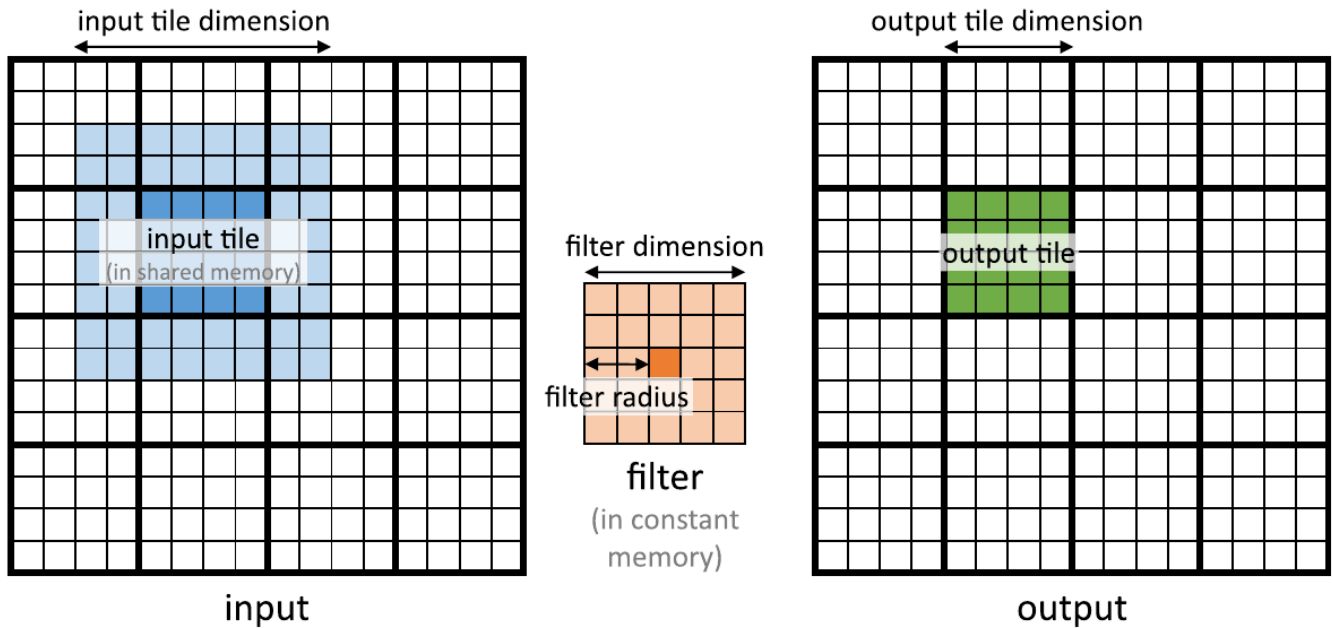


图7.11 二维卷积中的输入瓦片与输出瓦片。

我们将输入瓦片定义为计算输出瓦片中 P 元素所需的输入 N 元素集合。图 7.11 展示了对应于输出瓦片的输入瓦片（左侧的阴影区域）。请注意，为了确保它包括用于计算输出瓦片边缘处 P 元素所需的所有边界输入元素，输入瓦片的尺寸需要在每个方向上扩展滤波器的半径（在本例中为 2）。此扩展可以使输入瓦片比输出瓦片大得多。在这个玩具示例中，每个输出瓦片包含 $4 \times 2 = 16$ 个 P 元素，而每个输入瓦片包含 $(4 + 4)^2 = 8^2 = 64$ 个元素。在这种情况下，输入瓦片比输出瓦片大 3 个单位。但是，这种大比例是因为我们假设了一个小的输出瓦片维度，以便在玩具示例中更容易可视化。实际上，输出瓦片的维度会大得多，并且输入瓦片尺寸和输出瓦片尺寸之间的比率会接近 1.0。例如，如果输出尺寸为 $16 \times 16 = 256$ ，使用相同的 5×5 滤波器，输入瓦片尺寸将为 $(16 + 4)^2 = 400$ 。输入瓦片尺寸和输出尺寸之间的比率约为 1.6。虽然这个比率远小于 4，但它表明输入瓦片尺寸仍然可以大大大于输出瓦片，即使对于实际的输出瓦片尺寸也是如此。

在本节中，我们介绍一类瓦片卷积算法，其中所有块中的线程首先协作将输入瓦片加载到共享内存中，然后通过访问共享内存中的输入元素来计算输出瓦片的元素。这应该对读者听起来很熟悉；这个策略类似于在第 5 章《内存架构和数据局部性》中讨论过的瓦片矩阵乘法算法。主要的区别在于，第 5 章《内存架构和数据局部性》中的瓦片矩阵乘法算法假设输入瓦片与输出瓦片具有相同的维度，而卷积输入瓦片比输出瓦片大。这种输入瓦片大小与输出瓦片大小之间的差异增加了瓦片卷积核设计的复杂性。

为了解决输入瓦片大小与输出瓦片大小之间的差异，有两种简单的线程组织方式。第一种启动与输入瓦片大小匹配的线程块。这简化了输入瓦片的加载，因为每个线程只需要加载一个输入元素。然而，由于块的维度大于输出瓦片的维度，一些线程在计算输出元素时可能需要被禁用，这可能会降低执行资源利用率的效率。第二种方法启动与输出瓦片大小匹配的块。一方面，这种第二种策略使得输入瓦片加载更加复杂，因为线程需要迭代以确保加载所有输入瓦片元素。另一方面，它简化了输出元素的计算，因为块的维度与输出瓦片相同，并且在计算输出元素时不需要禁用任何线程。我们将基于第一种线程组织设计一个内核，并将第二种组织作为练习留给读者。

```

01 #define IN_TILE_DIM 32
02 #define OUT_TILE_DIM ((IN_TILE_DIM) - 2*(FILTER_RADIUS))
03 __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
04 __global__ void convolution_tiled_2D_const_mem_kernel(float *N, float *P,
05                                                     int width, int height) {
06     int col = blockIdx.x*OUT_TILE_DIM + threadIdx.x - FILTER_RADIUS;
07     int row = blockIdx.y*OUT_TILE_DIM + threadIdx.y - FILTER_RADIUS;
08     //loading input tile
09     __shared__ N_s[IN_TILE_DIM][IN_TILE_DIM];
10     if(row>=0 && row<height && col>=0 && col<width) {
11         N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
12     } else {
13         N_s[threadIdx.y][threadIdx.x] = 0.0;
14     }
15     __syncthreads();
16     // Calculating output elements
17     int tileCol = threadIdx.x - FILTER_RADIUS;
18     int tileRow = threadIdx.y - FILTER_RADIUS;
19     // turning off the threads at the edges of the block
20     if (col >= 0 && col < width && row >=0 && row < height) {
21         if (tileCol>=0 && tileCol<OUT_TILE_DIM && tileRow>=0
22             && tileRow<OUT_TILE_DIM){
23             float Pvalue = 0.0f;
24             for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
25                 for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
26                     Pvalue += F[fRow][fCol]*N_s[tileRow+fRow][tileCol+fCol];
27                 }
28             }
29             P[row*width+col] = Pvalue;
30         }
31     }
32 }

```

图7.12 使用常量内存的瓦片化二维卷积核对 F 进行操作。

图 7.12 展示了基于第一种线程组织的内核。每个线程首先计算其负责加载或计算的输入或输出元素的列索引 (col) 和行索引 (row) (行 06-07)。内核分配了一个与输入瓦片大小相同的共享内存数组 N_s (行 09)，并将输入瓦片加载到共享内存数组中 (行 10-15)。行 10 中的条件用于每个线程检查它正在尝试加载的输入瓦片元素是否为 ghost cell。如果是，则线程不执行内存加载，而是将零放入共享内存中。所有线程执行屏障同步 (行 15)，以确保整个输入瓦片在任何线程允许继续计算输出元素之前都已经在共享内存中。

现在，所有输入瓦片元素都在 N_s 数组中，每个线程都可以使用 N_s 元素计算它们的输出 P 元素值。请记住，输出瓦片比输入瓦片小，而块的大小与输入瓦片相同，因此每个块中只有一部分线程将用于计算输出瓦片元素。我们可以以多种方式选择用于此计算的线程。我们使用一种设计来停用 $FILTER_RADIUS$ 外部层的线程，如图 7.13 所示。

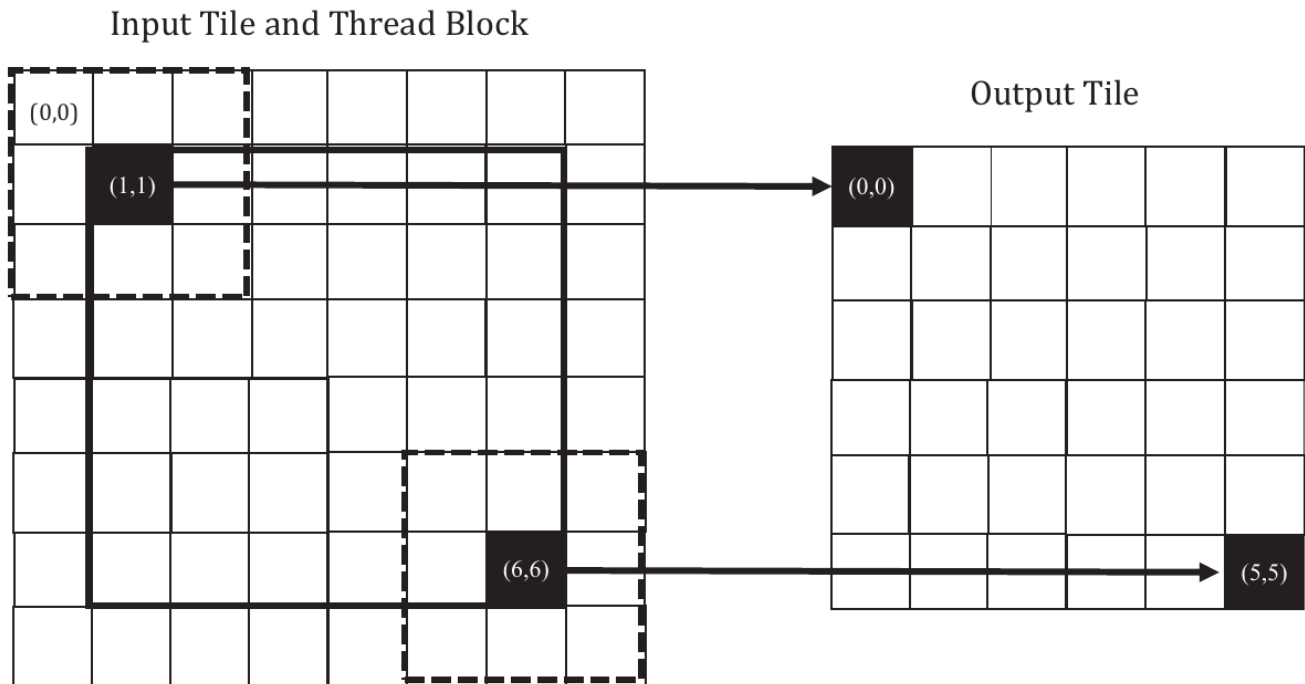


图7.13 一个小示例，说明了如何利用共享内存中的输入瓦片元素来计算输出瓦片元素的线程组织。

图 7.13 展示了使用 3×3 滤波器 ($\text{FILTER_RADIUS}=1$)、 8×8 输入瓦片、 8×8 块和 6×6 输出瓦片的卷积的小示例。图 7.13 的左侧显示了输入瓦片和线程块。由于它们的尺寸相同，它们被叠加在彼此上面。根据我们的设计，我们停用了 $\text{FILTER_RADIUS}=1$ 外部层的线程。图 7.13 左侧中央的粗线框圈起了用于计算输出瓦片元素的活动线程。在这个示例中，活动线程的 threadIdx.x 和 threadIdx.y 值都在 1 到 6 之间。

图 7.13 还展示了活动线程与输出瓦片元素的映射：活动线程 (tx, ty) 将使用输入瓦片元素的一个补丁来计算输出元素 $(tx - \text{FILTER_RADIUS}, ty - \text{FILTER_RADIUS})$ ，其中补丁的左上角是输入瓦片的元素 $(tx - \text{FILTER_RADIUS}, ty - \text{FILTER_RADIUS})$ 。这在图 7.12 的第 17-18 行中有所体现，其中列索引 (tileCol) 和行索引 (tileRow) 分别被赋值为 $\text{threadIdx.x} - \text{FILTER_RADIUS}$ 和 $\text{threadIdx.y} - \text{FILTER_RADIUS}$ 。

在图 7.13 中的我们的小示例中，线程 (1,1) 的 tileCol 和 tileRow 分别为 0 和 0。因此，线程 (1,1) 使用输入瓦片元素的一个 3×3 补丁来计算输出瓦片元素 (0,0)，该补丁在输入瓦片左上角的虚线框中被突出显示。图 7.12 的第 24-28 行中的 fRow-fCol 循环嵌套遍历该补丁并生成输出元素。块中的线程 (1,1) 将遍历补丁，其左上角是 $N_s[0][0]$ ，而线程 (5,5) 将遍历其左上角是 $N_s[5][5]$ 的补丁。

在第 06-07 行中， $\text{blockIdx.x} * \text{OUT_TILE_DIM}$ 和 $\text{blockIdx.y} * \text{OUT_TILE_DIM}$ 分别是块分配的输出瓦片开始处的水平和垂直 P 数组索引。正如我们之前讨论的， threadIdx.x-r 和 threadIdx.y-r 给出了对瓦片的偏移量。因此， row 和 col 变量提供了分配给每个活动线程的输出元素的索引。每个线程使用这两个索引在第 29 行中写入输出元素的最终值。

图 7.12 中的瓦片化二维卷积核比图 7.9 中的基本核心要长得多且更复杂。我们引入了额外的复杂性来减少对 N 元素的 DRAM 访问次数。目标是提高算术与全局内存访问比，以便实现的性能不受 DRAM 带宽的限制或更少受限制。回顾第 7.4 节，图 7.9 中的核心的算术与全局内存访问比为 0.5 OP/B。现在让我们为图 7.12 中的核心推导出这个比率。

对于处理数据边缘瓦片的块，处理虚 ghost 单元的线程不对这些 ghost 单元执行任何内存访问。这减少了这些块的内存访问次数。我们可以通过枚举使用每个 ghost 单元的线程数量来计算减少的内存访问次数。然而，对于大型输入数组，小的掩码尺寸的 ghost 单元的影响将不重要。因此，在计算瓦片卷积核的算术与全局内存访问比时，我们将忽略 ghost 单元的影响，只考虑那些内部线程块，其边界单元不是 ghost 单元。

现在我们计算图 7.12 中的瓦片核心的算术与全局内存访问比率。分配给输出瓦片元素的每个线程对于每个滤波器元素执行一次乘法和一次加法。因此，内部块中的线程共同执行 $\text{OUT_TILE_DIM}^2 \times (2 \times \text{FILTER_RADIUS} + 1)^2 \times 2$ 次算术操作。至于全局内存访问，所有全局内存访问都已转移到加载 N 元素到共享内存的代码中。分配给输入瓦片元素的每个线程加载一个 4 字节的输入值。因此每个内部块加载 $\text{IN_TILE_DIM}^2 \times 4 = (\text{OUT_TILE_DIM} + 2 \times \text{FILTER_RADIUS})^2 \times 4$ 字节。因此，瓦片核心的算术与全局内存访问比率为

$$\frac{\text{OUT_TILE_DIM}^2 \times (2 \times \text{FILTER_RADIUS} + 1)^2 \times 2}{(\text{OUT_TILE_DIM} + 2 \times \text{FILTER_RADIUS})^2 \times 4}$$

对于我们的示例，使用 5×5 的滤波器和 32×32 的输入瓦片 (28×28 输出瓦片)，比率为 9.57 OP/B。 32×32 的输入瓦片大小是当前 GPU 可以实现的最大尺寸。然而，我们可以对瓦片大小进行渐近分析，以得到对于此计算可实现的算术与全局内存访问比率的上限。如果 OUT_TILE_DIM 远大于 FILTER_RADIUS ，则我们可以将 $\text{OUT_TILE_DIM} + 2 \times \text{FILTER_RADIUS}$ 近似为 OUT_TILE_DIM 。这简化了表达式为 $(2 \times \text{FILTER_RADIUS} + 1)^2 \times 2 / 4$ 。这应该是一个非常直观的结果。在原始算法中，每个 N 元素都由大约 $(2 \times \text{FILTER_RADIUS} + 1)^2 \times 2$ 个线程多次加载，并且每个线程都对其执行两次算术操作。因此，如果瓦片大小无限大，并且每个 4 字节元素只加载到共享内存一次，那么比率应为 $(2 \times \text{FILTER_RADIUS} + 1)^2 \times 2 / 4$ 。

IN_TILE_DIM		8	16	32	Bound
5x5 filter (FILTER_RADIUS = 2)	OUT_TILE_DIM	4	12	28	-
	Ratio	3.13	7.03	9.57	12.5
9x9 filter (FILTER_RADIUS = 4)	OUT_TILE_DIM	-	8	24	-
	Ratio	-	10.13	22.78	40.5

图7.14 二维瓦片卷积的瓦片尺寸和滤波器尺寸作为函数的算术到全局内存访问比率。

图 7.14 展示了瓦片卷积核对于不同滤波器尺寸的算术与全局内存访问比率随着瓦片尺寸的变化，包括一个渐近上限。对于 5×5 滤波器，比率的上限为 12.5 OP/B。然而，实际可实现的比率在 32×32 的线程块大小限制下为 9.57 OP/B。对于较大的滤波器，如图 7.14 底部行中的 9×9 ，比率的上限为 40.5 OP/B。然而，在 32×32 的线程块大小限制下实际可实现的比率为 22.78 OP/B。因此，我们观察到较大的滤波器尺寸具有更高的比率，因为每个输入元素被更多线程使用。然而，较大的滤波器尺寸也具有更高的比率与实际实现比率之间的差距，因为较大数量的边界元素会导致较小的输出瓦片。

读者在使用小块和瓦片尺寸时应格外小心。它们可能导致的内存访问减少明显低于预期。例如，在图7.14中， 8×8 个块（输入瓦片）仅导致 5×5 滤波器的比率为 3.13 OP/B。在实践中，通常会使用较小的瓦片尺寸，因为芯片内存量不足，特别是在3D卷积中，芯片内存需求随着瓦片尺寸的增加而迅速增长。

7.5 使用缓存处理边缘单元的分块卷积

在图7.12中，代码的复杂性主要源于输入瓦片和块比输出瓦片大，这是由于加载边缘单元所导致的。回想一下，块的输入瓦片的边缘单元也是相邻瓦片的内部元素。例如，在图7.11中，输入瓦片的浅色阴影部分也是相邻块的输入瓦片的内部元素。这样，当一个块需要其边缘单元时，由于其相邻块的访问，这些边缘单元有很大可能已经在L2缓存中，因此可以从L2缓存中自然地提供对这些边缘单元的内存访问，而不会导致额外的DRAM流量。也就是说，我们可以将对这些边缘单元的访问保留在原始 N 元素中，而不是将它们加载到 N_{ds} 中。我们现在介绍一个使用相同尺寸的输入和输出瓦片，并且仅将每个瓦片的内部元素加载到共享内存中的分块卷积算法。


```

01  #define TILE_DIM 32
02  __constant__ float F_c[2*FILTER_RADIUS+1][2*FILTER_RADIUS+1];
03  __global__ void convolution_cached_tiled_2D_const_mem_kernel(float *N,
                                float *P, int width, int height) {
04      int col = blockIdx.x*TILE_DIM + threadIdx.x;
05      int row = blockIdx.y*TILE_DIM + threadIdx.y;
06      //loading input tile
07      __shared__ N_s[TILE_DIM][TILE_DIM];
08      if(row<height && col<width) {
09          N_s[threadIdx.y][threadIdx.x] = N[row*width + col];
10      } else {
11          N_s[threadIdx.y][threadIdx.x] = 0.0;
12      }
13      __syncthreads();
14      // Calculating output elements
15      // turning off the threads at the edges of the block
16      if (col < width && row < height) {
17          float Pvalue = 0.0f;
18          for (int fRow = 0; fRow < 2*FILTER_RADIUS+1; fRow++) {
19              for (int fCol = 0; fCol < 2*FILTER_RADIUS+1; fCol++) {
20                  if (threadIdx.x-FILTER_RADIUS+fCol >= 0 &&
21                      threadIdx.x-FILTER_RADIUS+fCol < TILE_DIM &&
22                      threadIdx.y-FILTER_RADIUS+fRow >= 0 &&
23                      threadIdx.y-FILTER_RADIUS+fRow < TILE_DIM) {
24                      Pvalue += F[fRow][fCol]*N_s[threadIdx.y+fRow][threadIdx.x+fCol];
25                  }
26                  else {
27                      if (row-FILTER_RADIUS+fRow >= 0 &&
28                          row-FILTER_RADIUS+fRow < height &&
29                          col-FILTER_RADIUS+fCol >= 0 &&
30                          col-FILTER_RADIUS+fCol < width) {
31                          Pvalue += F[fRow][fCol]*
32                          N[(row-FILTER_RADIUS+fRow)*width+col-
33                          FILTER_RADIUS+fCol];
34                      }
35                  }
36              }
37          }
38          P[row*width+col] = Pvalue;
39      }
40  }

```

应该是threadIdx.y - FILTER_RADIUS + fRow和
threadIdx.x - FILTER_RADIUS + fCol

图7.15 使用缓存处理边缘数据并利用常量内存处理F的分块二维卷积核。

【译注：代码有一点问题，请参看上面红色部分的说明。完整代码请参考[这里](#)。】

图7.15展示了一种使用缓存处理边缘单元的2D卷积核。在这个分块核中，共享内存N_ds数组只需保存瓦片的内部元素。因此，输入瓦片和输出瓦片具有相同的尺寸，即由常量TILE_DIM定义（第1行）。通过这种简化，N_s被声明为在x和y维度上都有TILE_DIM个元素（第6行）。

由于输入瓦片和输出瓦片的尺寸相同，线程块可以以相同的输入/输出瓦片尺寸启动。加载N_s元素变得更简单，因为每个线程可以简单地加载与其分配的输出元素具有相同x和y坐标的输入元素（第4-5行和第7-11行）。加载输入元素的条件也在第7行中简化了：由于核心不再将边缘单元加载到共享内存中，因此不存在加载虚拟单元的风险。因此，该条件只需要检查瓦片是否超出了输入数据的有效范围的常规边界条件。然而，计算P元素的循环体变得更加复杂。它需要添加条件来检查是否使用了边缘单元和虚拟单元。边缘单元的处理通过第17-20行中的条件进行，该条件检测输入元素是否落在输入瓦片的内部。如果是，则从共享内存中访问该元素。如果不是，则在第24-27行的条件中检查边缘单元是否为虚拟单元。如果是，则对该元素不进行任何操作，因为我们假设虚拟值为0。否则，从全局内存中访问该元素。读者应验证处理虚拟单元的条件与图7.7中使用的条件类似。

与图7.12中的核相比，图7.15中的核的微妙优势在于其块大小、输入瓦片大小和输出瓦片大小可以相同，并且可以是2的幂。由于图7.12中的核的输入瓦片大小和输出瓦片大小不同，因此在执行该核时可能会有更多的内存分歧和控制分歧。

7.6 总结

在本章中，我们研究了卷积作为一种重要的并行计算模式。虽然卷积在许多应用中被使用，比如计算机视觉和视频处理，但它也代表了许多并行算法的基础模式。例如，可以将偏微分方程求解器中的stencil算法视为卷积的一种特殊情况；这将是第8章“Stencil”的主题。另一个例子是，也可以将网格点力或势值的计算视为卷积的特殊情况，这将在第17章“迭代磁共振成像重建”中介绍。我们还将在第16章“深度学习”中应用本章学到的大部分内容到卷积神经网络中。

我们介绍了一种基本的并行卷积算法，其实现受限于DRAM带宽，用于访问输入和滤波器元素。然后，我们引入了常量内存和对核心代码和主机代码的简单修改，以利用常量缓存并消除对滤波器元素的几乎所有DRAM访问。我们进一步介绍了一种分块并行卷积算法，通过利用共享内存减少了DRAM带宽消耗，同时引入了更多的控制流分歧和编程复杂性。最后，我们介绍了一种使用L1和L2缓存处理边缘单元的分块并行卷积算法。

我们通过提升算术与全局内存访问比率来分析了分块的好处。这种分析是一项重要的技能，并且对于理解分块模式的好处非常有用。通过这种分析，我们可以了解小块尺寸的限制，特别是对于大滤波器和3D卷积而言，这一点尤为明显。

虽然我们只展示了1D和2D卷积的核心示例，但这些技术也可以直接应用于3D卷积。总体而言，由于维度更高，输入和输出数组的索引计算更加复杂。此外，对于每个线程，由于需要在加载瓦片和/或计算输出值时穿越多个维度，因此会有更多的循环嵌套。我们鼓励读者完成这些更高维度的核心作业练习。